

Design and Analysis of Serial Peripheral Interface for Automotive Controller

Izhar Izzudin bin Jamaludin
Intel Technology Sdn. Bhd.
Halaman Kampung Jawa,
Kawasan Perindustrian Bayan Lepas,
Penang.
izhar.izzuddin.jamaludin@intel.com

Hasliza binti Hassan
Department of Electronic Engineering
Faculty of Electrical and Electronic Engineering,
Universiti Tun Hussein Onn Malaysia,
Batu Pahat, Johor.
hliza@uthm.edu.my

Abstract—Serial Peripheral Interface (SPI) is a commonly used communication protocol that allows serial data transfer between a master and a slave device over a short distance. However, the current implementation of the Serial Peripheral Interface has a low speed and clock synchronization issue. When it comes to automotive, there are a few things that are needed to be taken into consideration to increase the performance of the interface device. The paper presents the design, simulation, verification and optimization of SPI based on automotive interface device specifications. This includes a higher speed for the interface device and the efficiency of power dissipation. The SPI has been successfully optimized with a 1000MHz speed, a power of 575.7694 μ W and a total area of 1102.81mm². The speed and area achieved are well improved over the current design of SPI. Even though the total dissipation achieved is above the current implemented SPI design, it is acceptable due to the technical relation between power and speed in VLSI design.

Keywords— Serial Peripheral Interface, speed, synchronization

I. INTRODUCTION

Initially, high-speed interfaces have been developed for high-end applications such as a communications server and control system. Synchronous transmission of data in full-duplex mode for short-distance serial communication can be enabled by using the Serial Peripheral Interface (SPI). Serial Peripheral Interface (SPI) technology was developed to replace parallel interfaces, so there is no need to route parallel bus around PCB. SPI also provides high-speed data transfer between the devices [1]. Serial Peripheral Interface protocol allowed serial communication between a master (host processor) and a slave device (peripherals). SPI is much faster than traditional Inter-Integrated Circuit (I2C) and currently is the most used communication protocol between peripheral devices and different IC [2].

System with no control over data transfer and confirmation that both data running at the same rate which called "asynchronous" (not synchronous) usually have common serial port such as transmit and receive lines. The problem arises when two systems have slightly different clocks try to communicate with each other because a computer depends on synchronized single "clock" [3].

A simple device-to-device digital interface for microcontroller, peripherals and sensor are needed to handle arbitrary-length data at high speed and eliminate the complex protocol-related programming tasks [4]. To meet these goals, the important is to include optimum performance to the power ratio. In this context, the problems arise in performing high-speed interface device such as:

1. The current implementation of the Serial Peripheral Interface has a speed problem, 500MHz [5].

2. The current Serial Peripheral Interface having a clock synchronization issue and inconsistency.

Therefore, this paper covered a solution for high-Speed Serial Peripheral Interface by analyse the speed, area and power of SPI in the automotive controller. The speed of SPI for an automotive controller at least 500 MHz [6]. The system cover most aspects of creating a high higher speed and low power requirements of data transfer using SPI for the automotive control system.

The SPI protocol architecture, implementation technique of the system, circuit schematic and simulation results will be discussed in the following sections.

II. SPI OPERATIONS

Traditional SPI is a serial communication protocol device that implements synchronous communication between master and slave. Any number of bits can be sent continuously. Master of SPI which is usually is a microcontroller while a slave is a sensor, display, memory chip, and many more. Many slaves can be connected to a single master of SPI. There are four main pins in SPI which are Slave select (SS), SCK (clock), MISO (Master in Slave out) and MOSI (master out slave in) pins.

The advantages of using SPI is that there is no start and stop bits which allow the data to stream continuously without any interruption. SPI also does not have a complicated slave addressing system compare to I2C. The transfer rate of SPI is also higher compared to another serial communication protocol such as UART and I2C. In addition to MOSI and MISO lines, two ways communication is possible because data can be sent and received at the same time. However, SPI is not as perfect as a serial communication device. This is because SPI required a minimum of four wires while other serial communication devices such as UART and I2C require only two wires. Next, SPI also does not have an error checking function like UART. Fig. 1 shows the basic connection topology between SPI master and slave.

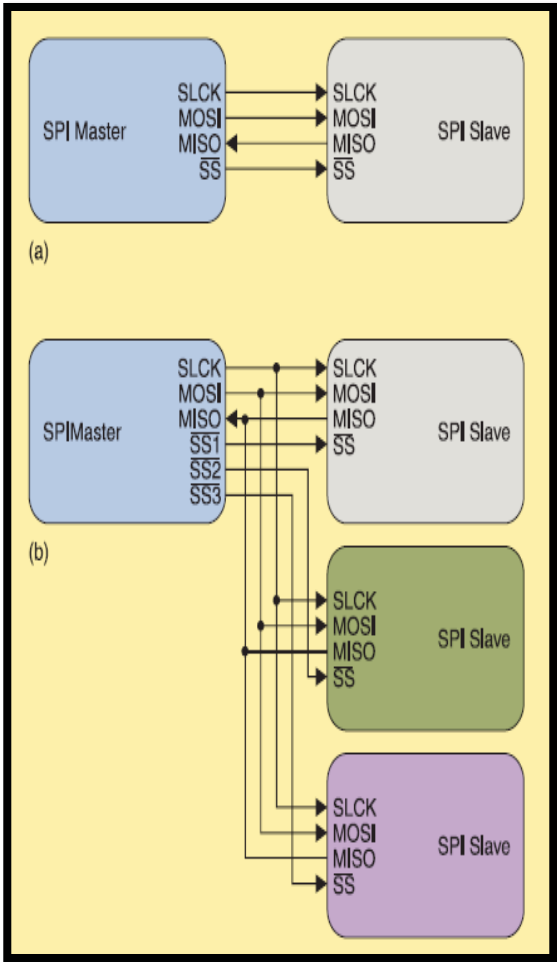


Fig. 1: a) SPI master connected to a single slave, b) SPI master connected to multiple slaves [7].

III. SPI DESIGN

SPI design is a process to determine the components and write a Verilog required to design a SPI. The components required are Xtal, Clock Logic, Receive Shift Register, Transmit Shift Register and Pin Control Logic. All of the components of the Verilog code are compiled using Synopsys software. SPI master mode in enable when MSTR = “1”, SS = “1” enable “1”, and SPE = “1”. SPI master also generates clock based on the combination of SPR0, SPR1 and SPI2X in SPI status register. If RST (RESET) is made zero, all the register also will be reset. During master mode, data from the received shift register is copied into transmit shift register through pin control logic. Pin control logic which contains DORD input will determine the data bit order of the data transfer process whether to transfer the data in normal or reversed condition. After the SS = “0”, transfer flag = “1” will be sent to the slave to initiate the data transfer and will transfer flag = “0” will stop the data transfer process. The received data from the salve will be stored into the transmit shift register from the received shift register. TABLE 1 shows the port description of the SPI design.

TABLE 1. SPI Port Description

PORT	MODE	DESCRIPTION
clk_div	in	Speed setting. The integer input is the number of system clocks per 1/2 period of sclk.
clock	in	System clock.
cpha	in	SPI clock phase setting.
cpol	in	SPI clock polarity setting.
enable	in	H: latches in settings, address, and data to initiate a transaction, L: no transaction is initiated.
miso	in	Master in, slave out data line.
mosi	out	Master out, slave in data line.
reset	in	Asynchronous active low reset.
sclk	buffer	SPI clock.
Ss	buffer	Slave select signals.

The constraint for the clock period is set to control the speed, which is 500MHz. The area constraint for the design also is set to the smallest possible in order to get the smallest possible area consumption that the DC can optimize. The analyze synthesis results for timing and congestion must be done to ensure the desired specification is achieved. All the data required by the physical design or layout tools generated by the Synopsys DC compiler.

Data transmission in SPI is full-duplex which is means both reception and transmission can be done at the same time. Both transmission and reception happen at the SPI data register. Every clock cycle, the data is shifted out one bit in a circular fashion. The communication starts with a clock configuration by the master to allow synchronous communication. Next, the master needs to select the slave that it wants to communicate by pulling the desired SS line low. At the end of the communication, the master deselects the slave by pulling the desired SS line high. The data is transfer from master to slave by using MOSI line and the data reception by the master is from MISO line. The number of clock cycle also will be given by the master after the clock output. In some cases of only data reception is required, the data in the master also will be shifted out to the slave because of how the transmission work (ring topology). There are four basic modes in SPI data transmission, Mode 0, Mode 1, Mode 2 and Mode 3 as in TABLE 2.

TABLE II. SPI Transfer Mode

MODE	IDLE CLOCK STATE	DATA CAPTURE	DATA WRITE
0	Low	rising edge	falling edge
1	Low	falling edge	rising edge
2	High	falling edge	rising edge
3	High	rising edge	falling edge

In this paper, the SPI protocol implement four logic signals: SCLK, MOSI (Master Output-Slave Input), MISO (Master Input-Slave Output), SS (Slave Select). SCLK is the clock, a unidirectional bus, which fed into the slave devices. MOSI is defined as output from master. MISO is defined as output from slave which is also known as serial data in. SS is an active low signal which is used to select the slave devices. A full duplex data transmission is occurred in SPI clock cycle.

SPI consists of two modes, which are the master mode and the slave mode. The CLPD generated the clock signal (SCK) during the master mode while during the slave mode, the clock was input by the peripheral. Mode determined by SS bit when SS= '1', the CLPD is in master mode while SS='0' the CLPD is in slave mode. From then on, data transfer is conducted by the SPI status register, SPI data register and SPI control register. Three main registers in the SPI design which are control register, a data register and status register. The control register is used to control the data transfer operation while the data register uses to store the data during the transfer operation.

Next, the clock of the SPI is determined by the status register. In addition to that, there are five main components of the SPI which are xtal, clock logic, receive shift register, transmit shift register and pin control logic. The complete design of SPI Master is enabled and generated output only when an active high MSTR '1' signal is presented. The inputs for the simulation were set as an active high MSTR, SPE was set to '1', ENABLE was set to '1', DORD was set to '0', CPHA and CPOL were set to '0' and DATALOAD was set to '1'.

IV. RESULT AND DICUSSION

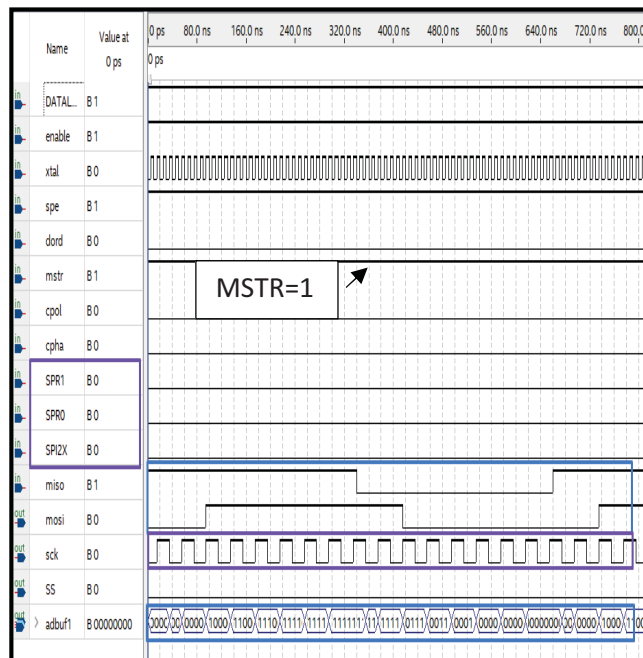


Fig. 2: Result of SPI output

Based on the results shows in Fig. 2, the output waveform for the Serial Clock was divided by four based on the input clock (xtal). This because the combination of logic SPI2X, SPR1 and SPR0 [000]. The output waveform for MOSI has loaded the value in MISO. Other than that, the value of output ADBUF received the input data in MISO. The value of MOSI

and ADBUF were the same. The output waveform shows that the SPI Master produced a correct output based on the input given. The functional simulation, rcv_shift_register based on the condition of CPOL. The signal for CPOL is set to '1'. The inputs of the clock were set with a 10ns period, "mstr" was set to active high '1', "reset" was set to '1' and "miso" was set to '1' in input. Fig. 3 shows final data transfer of SPI. Data out print the value of ADBUF as data input with control by CPOL.

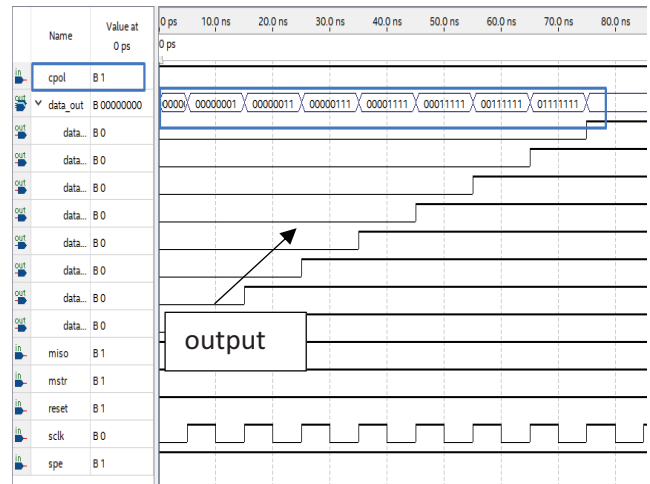


Fig. 3: Result of SPI simulation

The result of the performance analysis was synthesis for 3 clocks tabulated which is 1ns, 2ns and 3ns for speed, area and power.

TABLE 3. DC Timing Analysis

Clock Period (ns)	1ns	2ns	3ns
Delay -max (setup)	0.18	1.18	2.19
Delay -min (hold)	0.19	0.19	0.19
Area (mm ²)	1081.95	1080.94	1079.92

Based on the result obtained in TABLE 3, the value of delay -max, which is for the setup violation check time of design is acceptable. It is meant that the Data Arrival Time is less than the Data Required Time. The Data Arrival Time is the time taken by the data to arrive at a certain point. Besides, The Data Required Time is a total time that the circuit needs to function at the desired clock period or speed. The setup time value is increased when the value of the clock period increased. This is because of the difference between Data Required Time and Data Arrival Time setup time increased.

In addition, the value of delay -min, which is for the hold violation check time of design is also acceptable. It is meant that the Data Arrival Time for the hold time is more than the Data Required Time. The Data Arrival Time is the time taken by the data to arrive at a certain point. Besides, The Data Required Time is a total time that the circuit needs to transfer the input to the output of the design at the desired clock period or speed. The number of delays -min for the three clock period were maintained.

The combination of these modules produces an SPI master design that constrained at 1ns which equals 1000MHz speed. The gate-level simulation after the synthesise process successfully output the same waveform as before the synthesis process began. It indicates that the SPI master is still functioning correctly after the synthesis process. The area obtained from the final design is 1102.81 μm^2 which is also smaller than the current SPI design as in Fig. 4.



Fig. 4: Global Route Congestion

The place and route of the design have been successfully optimized for a total of power of 575.7694 μW . It can conclude that the speed of the SPI design is increased, and the total area used by the cell is decreased which technically improved the performance of the current SPI. However, this paper produces more power dissipation due to the increased speed of the design. The power that has been obtained is acceptable due to the technical relation between power and speed. Therefore, this paper was conducted successfully.

V. CONCLUSION

In this paper, an implementation of SPI with analysis of power, area and speed is presented. The modules are designed and simulated with Verilog HDL. The SPI has been successfully optimized with 1000MHz speed, power dissipation 575.7694 μW with total area 1102.81mm². The speed and area achieved are well improved over the current design of SPI. This SPI control bus architecture can enable the industrial fabrication of chip in a way where only a pressing of one switch can test itself. It would save valuable time and cost of testing circuits significantly.

ACKNOWLEDGMENT

Authors wish to express their gratitude to Universiti Tun Hussein Onn Malaysia for sponsorship this paper.

REFERENCES

- [1] Kamal Prakash Pandey, A. (2018). Design and Implementaion of SPI Protocol. *International Journal of Advance Engineering and Research Development*, 4(12), 1010–1016.
- [2] Choudhury, S., G.K.Singh, & R.M.Mehra. (2014). Design and Verification Serial Peripheral Interface (SPI) Protocol for Low Power Applications. *International Journal of Innovative Research in Science, Engineering and Technology*, 03(10), 16750–16758.
- [3] Frenzel, L. E. (2016). Serial Peripheral Interface (SPI). In *Handbook of Serial Communications Interfaces* (pp. 143–145). <https://doi.org/10.1016/b978-0-12-800629-0.00035-8>.
- [4] Pini, A. (2019). SPI Simplifies Device Communication. Retrieved May 20, 2019, from <https://www.digikey.com/en/articles/techzone/2019/feb/why-how-to-use-serial-peripheral-interface-simplify-connections-between-multiple-devices>.
- [5] Deepak Siddharth, P. (2018). UVM Verification of an SPI Master Core. *Rochester Institute of Technology*.
- [6] Jandiyam, R., Kandi, S. R., & Mohammad, U. F. (2017). Design and Implementation of SPI Module in Verilog HDL using FPGA Design Flow. *International Journal of Engineering Development and Research*, 5(4), 1090–1094.
- [7] Leens, F. (2009, February). An introduction to I2C and SPI protocols. *IEEE Instrumentation and Measurement Magazine*, Vol. 12, pp. 8–13. <https://doi.org/10.1109/MIM.2009.476294>.